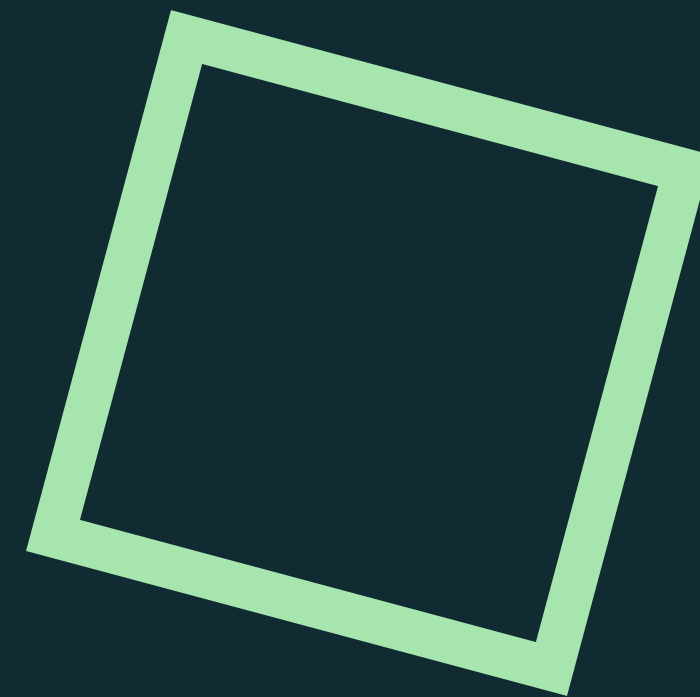


认识 AI 从模型到工具

理解技术格局，建立判断力。用技术语言，把 AI 变成协作工具。



01

这门课，沿着两条主线展开

一个解决认知，一个走向实践；两者彼此促进。

技术认知

看清 AI 的技术结构，回应焦虑与迷茫。

效能工具

选择合适的编程工具，掌握使用方式。

人的角色

理解程序员如何提出需求、判断结果。

先归类，再理解新名词

从技术名词转向技术结构

新概念再多，也先问两句： 它在哪一层？ 它起什么作用？

把 AI 放进一张分层地图

从底层能力，到直接面向用户的工具。



裸模型：能力与内部机制

模型名字只是入口，内部机制也属于这一层。

- 01 GPT-5.2、Kimi、GLM、Gemini
- 02 注意力机制、KV-Cache
- 03 上下文窗口，以及模型微调



按原课程的模型与术语示例呈现。

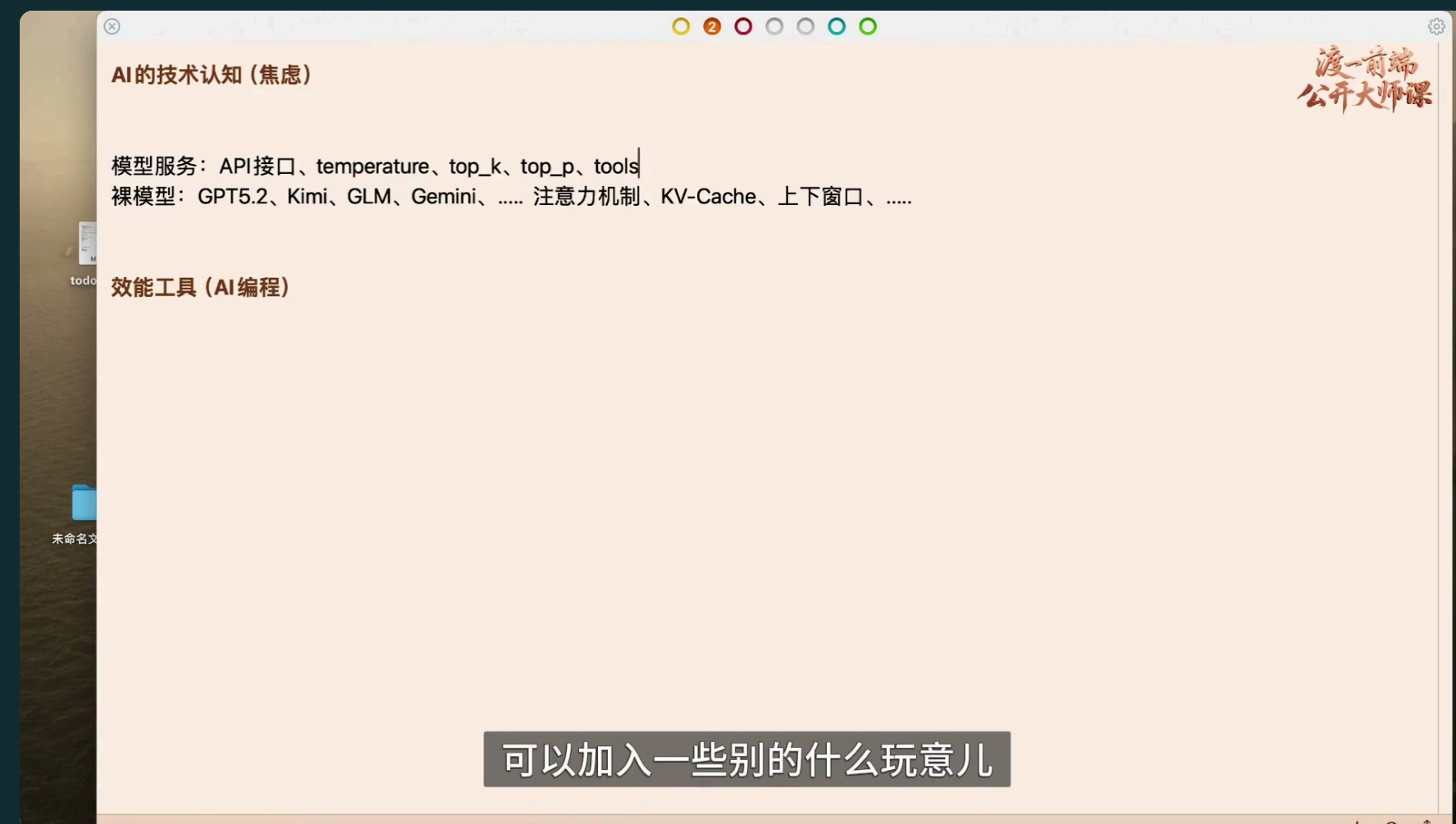
模型服务：把能力开放给程序

在裸模型基础上封装，常见入口是 API。

01 通过远程接口发送请求、获取回复

02 服务层参数：temperature、top_k、top_p

03 tools 等能力，也在这一层理解



API 参数与模型内部机制，需要分层理解。

学习重点，随目标而变

先确定要做什么，再决定深入哪一层。

使用工具

重点理解应用层及具体使用方式。

开发应用

模型服务层 + 常见应用开发手段。

微调模型

进入裸模型层，理解模型本身。

同样的工具，效果为何不同？

课程用 Vue、React 的学习差异作类比。

01 认知深度会影响结果

02 知识盲区会影响判断与使用

03 理解技术，也能缓解未知带来的焦虑

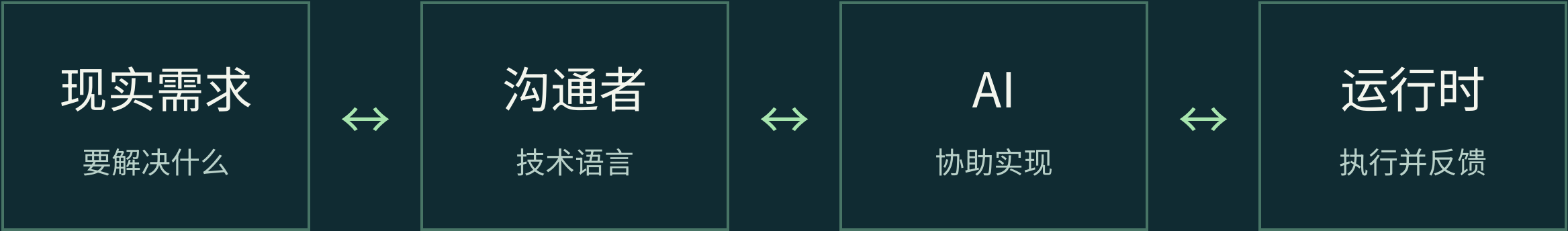
程序员一直在承担沟通角色

把需求变成可执行的东西，再根据结果调整。



AI 加入链路，桥梁仍然重要

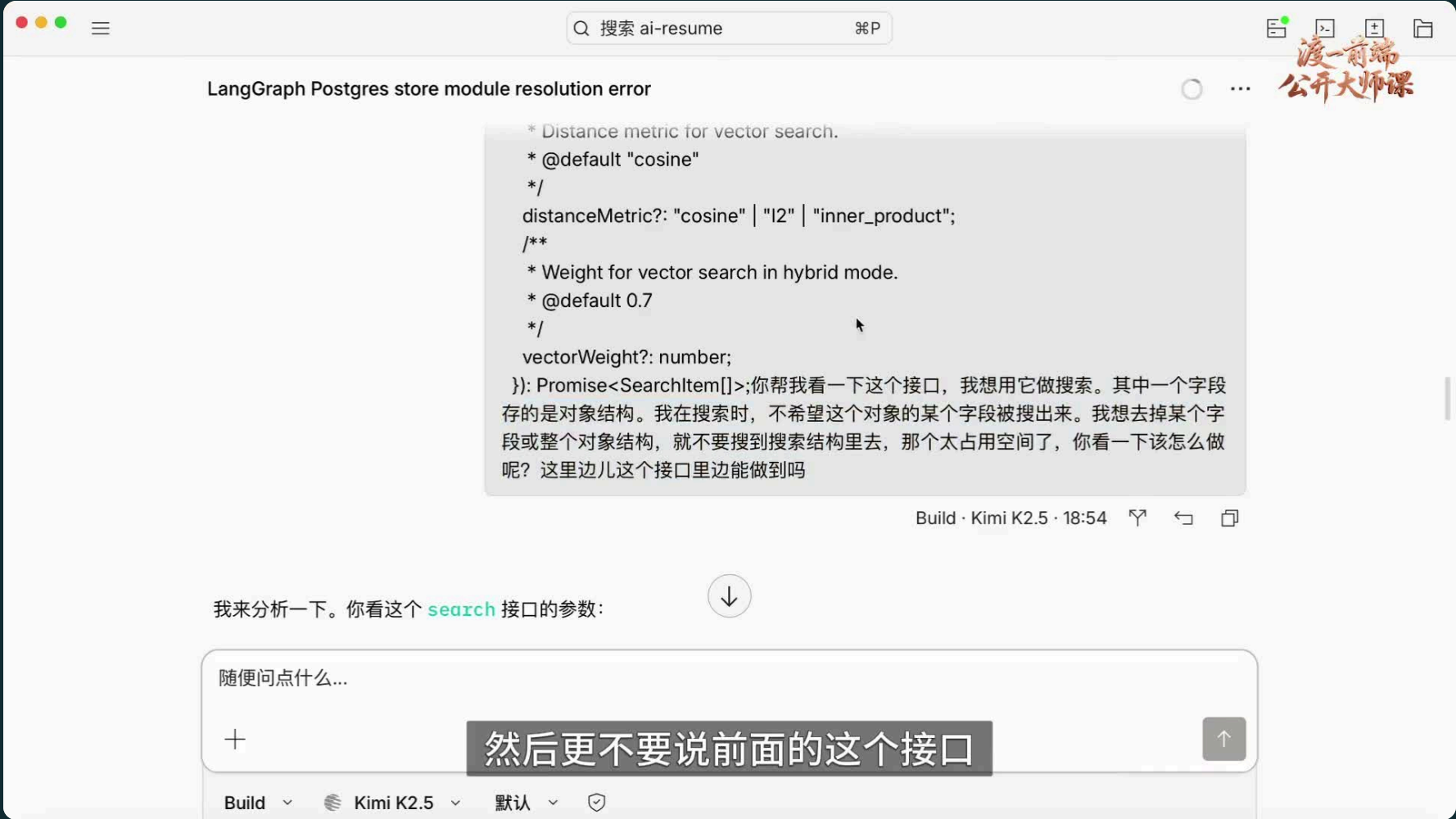
变化在协作方式，判断需求与结果的工作仍然存在。



案例一：问题要说到技术层面

看起来是自然语言，内容却依赖技术理解。

- 01 把接口定义和背景交给 AI
- 02 说清字段、对象结构与搜索需求
- 03 判断现有 API 能否支持目标



搜索时排除某个子字段或对象结构，降低不必要的结果体积。

案例二：测试通过 ≠ 功能正确

识别“看似修好，实际改坏”，需要理解原有行为。

- 01 AI 修改了项目里的过滤器
- 02 讲者在过程中发现功能被影响
- 03 要求修复报错，同时保留正确功能

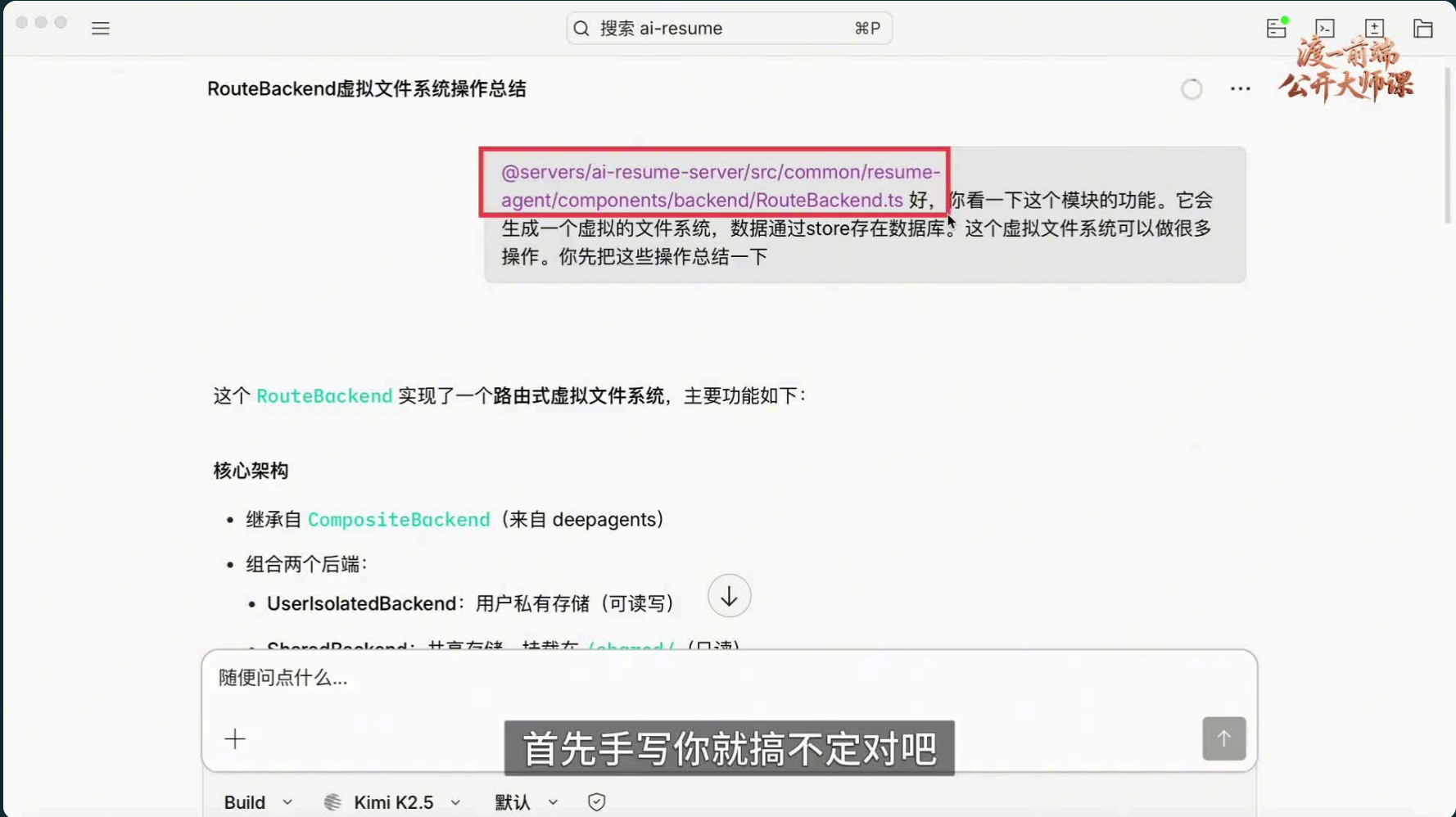


项目约定还包括文件创建、更新、删除的存在性处理。

案例三：用关键上下文引导阅读

模型不一定熟悉新库；代码与关键提示共同提供依据。

- 01 指明 RouteBackend.ts 模块
- 02 交代虚拟文件系统与 store 存储
- 03 让 AI 阅读逻辑，再总结操作与文档



本课展示的是沟通方式，没有展开完整后端实现。

案例四：懂技术，才能提出疑问

“有疑问”并不等于“代码一定写错了”。

- 01 发现不熟悉的返回对象结构
- 02 追问 `__message__` 与 `__data__` 的含义
- 03 通过解释确认设计，而非盲目接受



原案例中讲者确认代码没有问题，重点在于提出和验证疑问。

商业项目，需要持续沟通

简单小游戏与大型商业应用，不能混为一谈。

01 原课举例：几万、几十万，甚至上百万行代码

02 一句话难以交代所有需求与约束

03 理解越深广，指令、方向和步骤越准确

能力要求，发生的是重心变化

以下是课程讲者的观察与论证。

具体实施

部分手敲代码、API 用法记忆的负担降低。

技术判断

判断需求、功能、实现与测试是否一致。

认知体系

思维、知识广度与深度，更影响协作效果。

理解 AI，才能更好地使用 AI

技术认知 × 工具实践

知道它擅长什么、哪里容易出错，才能选择合适的工具，给出恰当的指令与反馈。